



CyberDefenders

RESOLUCIÓN DE LABORATORIO

Lab: Ransomed (CyberDefenders)

Análisis estático y dinámico de un ejecutable PE32 malicioso. El laboratorio se enfoca en revertir el empaquetado en memoria, revelar técnicas de ofuscación (Stack Strings) e interceptar APIs de Windows para identificar la inyección de código mediante Process Hollowing.

Sebastian David
Torres Reyes

Escenario:

A cybersecurity team received an alert about suspicious memory activity on a company workstation. The security monitoring system flagged an unknown executable exhibiting high entropy values, indicative of a packed or obfuscated binary. Further investigation revealed that the malware dynamically allocated memory, injected shellcode, and executed it via indirect jumps, suggesting an unpacking routine.

Your task is to analyze the malware sample using static and dynamic analysis techniques. Examine the PE structure, entropy levels, and memory allocations to determine how the malware unpacks itself, resolves API functions, and executes its payload.

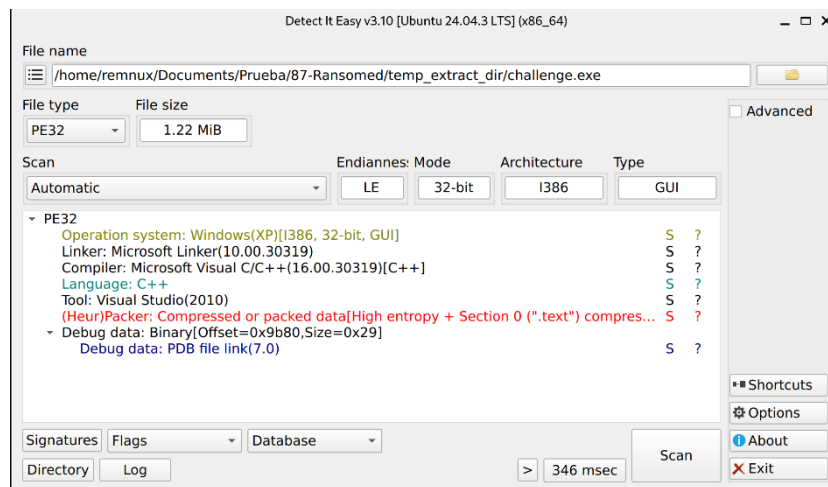
Herramientas utilizadas:

- Detect It Easy (DIE)
- PEStudio
- x32dbg
- scdbg
- MITRE ATT&CK

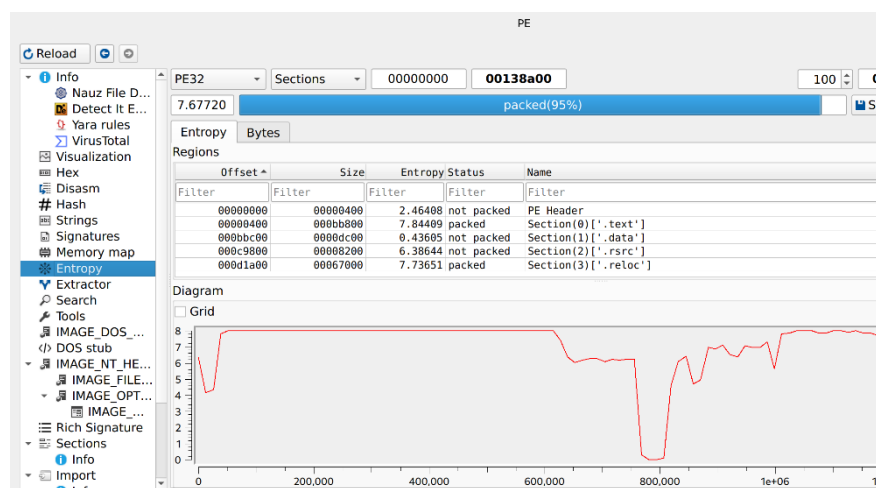
P1: What is the value of entropy?

R: 7.677

Se utilizó la herramienta Detect It Easy (DIE) para extraer información del archivo sospechoso, determinando que se trata de un ejecutable PE32 (Portable Executable) compatible con el sistema operativo Windows. Fue compilado mediante Microsoft Visual C/C++ y está escrito en lenguaje C++ a través del entorno de Visual Studio. El motor heurístico alertó sobre una alta entropía en la sección .text, lo que indica la presencia de datos comprimidos o empaquetados.



Al realizar un análisis avanzado, se determinó una entropía global de 7.677, lo que confirma que el 95% del archivo se encuentra empaquetado. Al desglosar las regiones del archivo, se identificó que las secciones .text y .reloc presentan datos ofuscados (alta entropía), mientras que el resto de las secciones (como la cabecera PE) no lo están. Esto representa un fuerte indicador de anomalía y sugiere que ambas secciones están siendo manipuladas para ocultar código malicioso o shellcode.



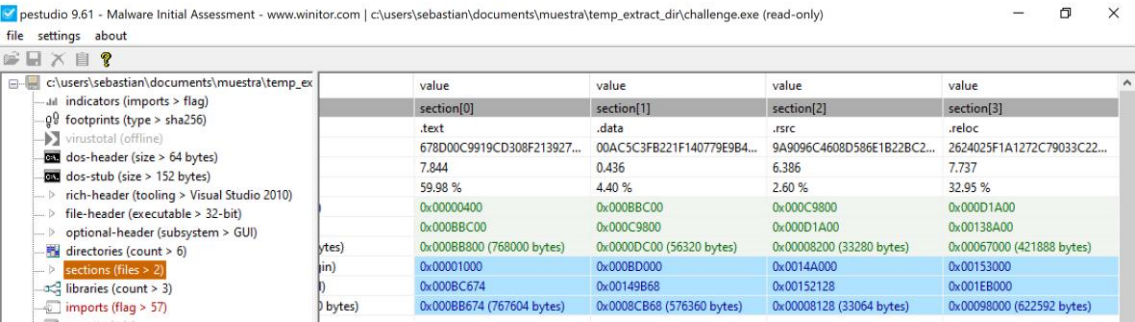
P2: What is the number of sections?

R: 4

El archivo malicioso contiene cuatro secciones: .text, .data, .rsrc y .reloc.

Regions				
Offset ^	Size	Entropy	Status	Name
Filter	Filter	Filter	Filter	Filter
00000000	00000400	2.46408	not packed	PE Header
00000400	000bb800	7.84409	packed	Section(0)['.text']
000bbc00	0000dc00	0.43605	not packed	Section(1)['.data']
000c9800	00008200	6.38644	not packed	Section(2)['.rsrc']
000d1a00	00067000	7.73651	packed	Section(3)['.reloc']

Mediante PESTudio, se pudo verificar individualmente la alta entropía de las secciones .text (7.844) y .reloc (7.737), lo que confirma el empaquetado u ofuscación de estos segmentos. Además, la sección .rsrc (6.386) también presenta una entropía elevada, aunque menor a las mencionadas anteriormente, lo que podría indicar también manipulación u ocultamiento de payloads adicionales dentro de los recursos del sistema.



value	value	value	value
section[0]	section[1]	section[2]	section[3]
.text	.data	.rsrc	.reloc
678D00C9919CD308F213927...	00AC5C3FB221F140779E9B4...	9A9096C4608D586E1B22BC2...	2624025F1A1272C79033C22...
7.844	0.436	6.386	7.737
59.98 %	4.40 %	2.60 %	32.95 %
0x00000400	0x000BB800	0x000C9800	0x000D1A00
0x000BB800	0x000C9800	0x000D1A00	0x00138A00
0x000BB800 (768000 bytes)	0x000D1A00 (56320 bytes)	0x000C9800 (33280 bytes)	0x000D1A00 (421888 bytes)
0x00001000	0x000BD000	0x0014A000	0x00153000
0x000BC674	0x00149B68	0x00152128	0x001EB000
0x000BB674 (767604 bytes)	0x0008CB68 (576360 bytes)	0x0008128 (33064 bytes)	0x00098000 (622592 bytes)

P3: What is the name of the technique used to obfuscate string?

R: Stack Strings

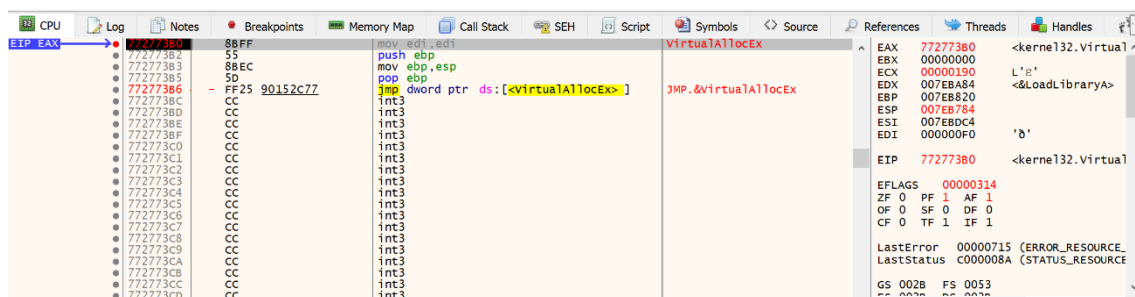
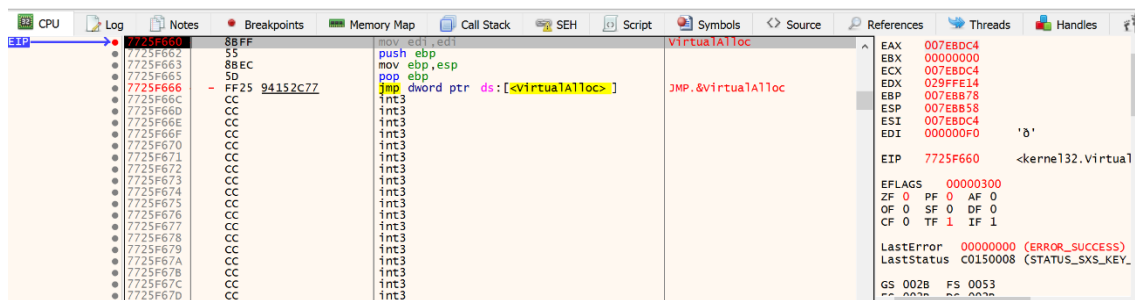
Al depurar el malware mediante x32dbg, se determinó que el binario está protegido casi en su totalidad por una técnica de Empaquetado (Runtime Packing), confirmando la alta entropía detectada previamente por DIE. El malware cuenta con un motor (stub) que se descifra a sí mismo en memoria mediante un bucle de ejecución antes de ceder el control al código real. Adicionalmente, dentro del código se identificó el uso de Stack Strings para ocultar cadenas de texto específicas directamente en la pila de la memoria durante su ejecución.

P4: What is the API that used malware-allocated memory to write shellcode?

R: VirtualAlloc

Comúnmente, para la creación de shellcode, un malware asigna un espacio en memoria utilizando la librería kernel32.dll y APIs, ya sea para la asignación de espacio en su propio proceso (VirtualAlloc) o mediante la inyección en otros procesos legítimos (VirtualAllocEx).

Con base en ello, se establecieron dos breakpoints en x32dbg (bp VirtualAlloc y bp VirtualAllocEx) con el objetivo de determinar qué APIs llama el malware y el proceso que sigue para la asignación de memoria. Como resultado, se observó que el malware emplea ambas funciones; sin embargo, ejecuta prioritariamente VirtualAlloc al ser la primera API en llamarse.



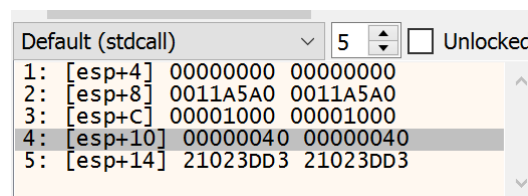
P5: What is the protection of allocated memory?

R: ERW

Cuando se habla de protección de memoria asignada, se hace referencia a los permisos configurados para dicha región. Tras establecer un breakpoint en la función VirtualAlloc, se detectó que el malware la invoca hasta tres veces (posiblemente para alojar componentes secundarios). Por lo tanto, el segundo parámetro (dwSize) es clave para identificar qué llamada corresponde al payload principal debido al tamaño de la memoria solicitada.

1er VirtualAlloc:

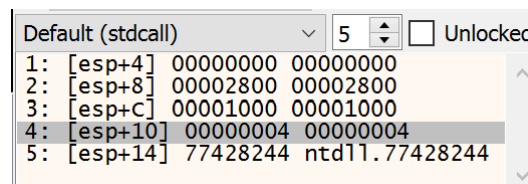
- lpAddress (0x0): La dirección inicial deseada para la región de memoria es un valor de NULL, es decir, el sistema operativo elige la dirección automáticamente.
- dwSize (0x11A5A0): El espacio de memoria que se va asignar es 1 156 512 bytes.
- flAllocationType (0x1000): Corresponde a la constante MEM_COMMIT. Indica al sistema operativo que asigne espacio de almacenamiento físico (en RAM o en el archivo de paginación) e inicialice la memoria en ceros de forma inmediata.
- flProtect (0x40): Corresponde a la constante PAGE_EXECUTE_READWRITE. Permite el acceso de ejecución, lectura y escritura a la región de páginas asignada (comportamiento típico para inyección de shellcode).



Default (stdcall)		5	☐ Unlocked
1:	[esp+4]	00000000	00000000
2:	[esp+8]	0011A5A0	0011A5A0
3:	[esp+C]	00001000	00001000
4:	[esp+10]	00000040	00000040
5:	[esp+14]	21023DD3	21023DD3

2do VirtualAlloc:

- lpAddress (0x0): La dirección inicial deseada para la región de memoria es un valor de NULL, es decir, el sistema operativo elige la dirección automáticamente.
- dwSize (0x2800): El espacio de memoria que se va asignar es 10 240 bytes.
- flAllocationType (0x1000): Corresponde a la constante MEM_COMMIT. Indica al sistema operativo que asigne espacio de almacenamiento físico (en RAM o en el archivo de paginación) e inicialice la memoria en ceros de forma inmediata.
- flProtect (0x4): Corresponde a la constante PAGE_READWRITE. Permite el acceso de lectura y escritura a la región de páginas asignada, pero prohíbe explícitamente la ejecución de código.



Default (stdcall)		5	☐ Unlocked
1:	[esp+4]	00000000	00000000
2:	[esp+8]	00002800	00002800
3:	[esp+C]	00001000	00001000
4:	[esp+10]	00000004	00000004
5:	[esp+14]	77428244	ntd11.77428244

3er VirtualAlloc:

- lpAddress (0x0): La dirección inicial deseada para la región de memoria es un valor de NULL, es decir, el sistema operativo elige la dirección automáticamente.
- dwSize (0x4): El espacio de memoria que se va asignar es 4 096 bytes.
- flAllocationType (0x1000): Corresponde a la constante MEM_COMMIT. Indica al sistema operativo que asigne espacio de almacenamiento físico (en RAM o en el archivo de paginación) e inicialice la memoria en ceros de forma inmediata.
- flProtect (0x4): Corresponde a la constante PAGE_READWRITE. Permite el acceso de lectura y escritura a la región de páginas asignada, pero prohíbe explícitamente la ejecución de código.

Default (stdcall)	5	☐ Unlocked
1: [esp+4]	00000000	00000000
2: [esp+8]	00000004	00000004
3: [esp+C]	00001000	00001000
4: [esp+10]	00000004	00000004
5: [esp+14]	77428244	ntdll.77428244

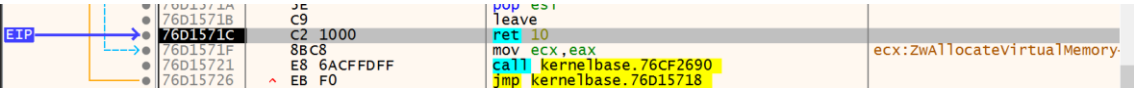
Como se mencionó anteriormente, el malware también invoca la API VirtualAllocEx lo que indica que tras la ejecución del shellcode, el malware se prepara para la inyección de procesos legítimos y asignación de espacio en memoria.

VirtualAllocEx:

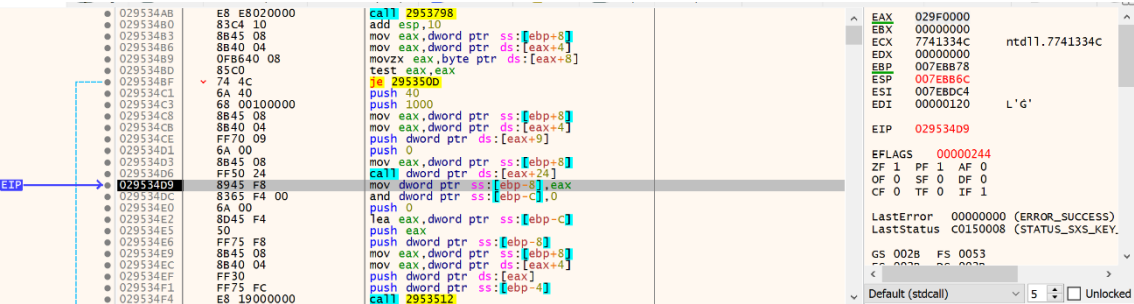
- HProcess (asignación dinámica): El malware apunta a un proceso externo en ejecución para realizar la inyección.
- lpAddress (0x400000): La dirección de memoria exacta donde se solicitará la creación de espacio dentro del proceso remoto es en 0x400000.
- dwSize (0x137000): El espacio de memoria que se va asignar es 1 273 856 bytes.
- flAllocationType (0x3000): El valor 0x3000 resulta de la combinación de dos constantes esenciales de Windows: MEM_COMMIT (0x1000) y MEM_RESERVE (0x2000). Esto le ordena al sistema operativo apartar la dirección y asignarle espacio físico en la RAM de forma simultánea.
- flProtect (0x40): Corresponde a la constante PAGE_EXECUTE_READWRITE. Permite el acceso de ejecución, lectura y escritura a la región de páginas asignada.

Default (stdcall)	5	☐ Unlocked
1: [esp+4]	000001B0	000001B0
2: [esp+8]	00400000	challenge.004000
3: [esp+C]	00137000	00137000
4: [esp+10]	00003000	00003000
5: [esp+14]	00000040	00000040

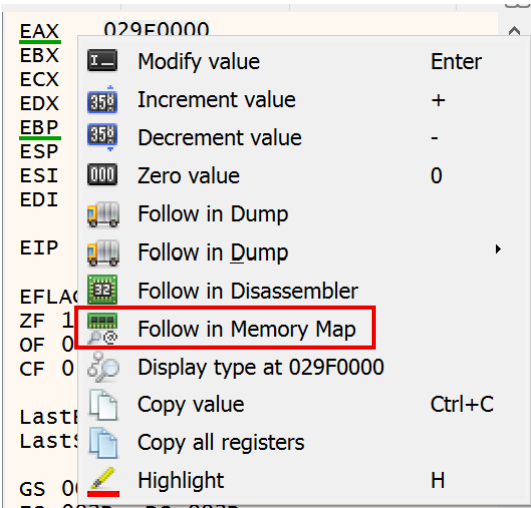
En base a lo hallado, la primera invocación de la API VirtualAlloc detalla un espacio de 1 156 512 bytes requeridos lo que correspondería al payload principal por lo cual se realizó un seguimiento de dicha invocación mediante CTRL+F9 para interceptar el instante en el que la API termina su función de asignación (RET).



Posteriormente, se utilizó F8 (Step Over) para posicionarse en un instante después de la asignación de memoria. Se halló que la dirección de memoria asignada para el shellcode fue 029F0000 (dirección asignada dinámicamente).



Se realizó un seguimiento de la dirección de memoria asignada en Memory Map donde se confirmaron los permisos configurados (protección) de la memoria asignada ERW (Execute, Read, Write).



Address	Size	Party	Info	Content	Type	Protection	Initial
02940000	00005000	User	Heap (ID 1)		PRV	-RW--	-RW--
02945000	0000B000	User	Reserved (02940000)		PRV	-RW--	-RW--
02950000	00003000	User	Reserved		PRV	-RW--	-RW--
02953000	00091000	User			PRV	ERW--	-RW--
029E4000	00001000	User	Reserved (02950000)		PRV	-RW--	-RW--

P6: What assembly instruction is used to transfer execution to the shellcode?

R: `jmp dword ptr ss:[ebp-4]`

Cabe recalcar que la dirección de memoria asignada no siempre será la misma en cada ejecución debido a la asignación dinámica del sistema operativo cuando el primer parámetro (lpAddress) de la API VirtualAlloc se configura en NULL.

Tras obtener la dirección de memoria asignada (por ejemplo, 02BC0000) guardada en el registro EAX, se realizó un seguimiento de las instrucciones para identificar la transferencia de ejecución:

- En 028CF4D9, el malware copia la dirección de memoria virgen (alojada en EAX) dentro de la variable local [ebp-8].
- Posteriormente, entre 028CF4E6 y 028CF4F1, el código apila el contenido de [ebp-8] y otras variables locales para pasarlas como parámetros.
- En 028CF4F4 se ejecuta una instrucción CALL, la cual utiliza los parámetros previamente apilados para desempaquetar y escribir el shellcode dentro de la dirección de memoria asignada.
- Una vez escrito el payload, en 028CF4FC y 028CF4FF el malware recupera la dirección guardada en [ebp-8] y la mueve a la variable [ebp-4].
- Finalmente, en 028CF50D se ejecuta la instrucción `jmp dword ptr ss:[ebp-4]`, la cual lee la dirección que contiene el shellcode y realiza el salto hacia ella (02BC0000) para ejecutar la carga útil.

0282E4D1	6A 00	push 0	EAX	028C0000
0282E4D3	8B45 08	mov eax,dword ptr ss:[ebp+8]	EBX	00000000
0282E4D6	FF50 24	call dword ptr ds:[eax+24]	ECX	7741334C ntdll.7741334C
0282E4D9	8945 F8	mov dword ptr ss:[ebp-8],eax	EDX	00000000
0282E4DC	8365 F4 00	and dword ptr ss:[ebp-C],0	EBP	007EBB78
0282E4E0	6A 00	push 0	ESP	007EBB6C
0282E4E2	8D45 F4	lea eax,dword ptr ss:[ebp-C]	ESI	007EBDC4
0282E4E5	50	push eax	EDI	00000120 L'G'
0282E4E6	FF75 F8	push dword ptr ss:[ebp-8]	ETP	0282E4D9
0282E4E9	8B45 08	mov eax,dword ptr ss:[ebp+8]	EFLAGS	00000246
0282E4EC	8B40 04	mov eax,dword ptr ds:[eax+4]	ZF	1 PF 1 AF 0
0282E4EF	FF30	push dword ptr ds:[eax]	OF	0 SF 0 DF 0
0282E4F1	FF75 FC	push dword ptr ss:[ebp-4]	CF	0 TF 0 IF 1
0282E4F4	EB 19000000	call 028E512	LastError	00000000 (ERROR_SUCCESS)
0282E4F9	83C4 14	add esp,14	LastStatus	C0150008 (STATUS_SXS_KEY)
0282E4FC	8B45 F8	mov eax,dword ptr ss:[ebp-8]	GS	0028 FS 0053
0282E4FF	8945 FC	mov dword ptr ss:[ebp-4],eax		
0282E502	8B45 08	mov eax,dword ptr ss:[ebp+8]		
0282E505	8B40 04	mov eax,dword ptr ds:[eax+4]		
0282E508	8B40 F4	mov ecx,dword ptr ss:[ebp-C]		
0282E50B	8908	mov dword ptr ds:[eax],ecx		
0282E50D	FF65 FC	jmp dword ptr ss:[ebp-4]		

P7: What is the number of functions the malware resolves from kernel32?

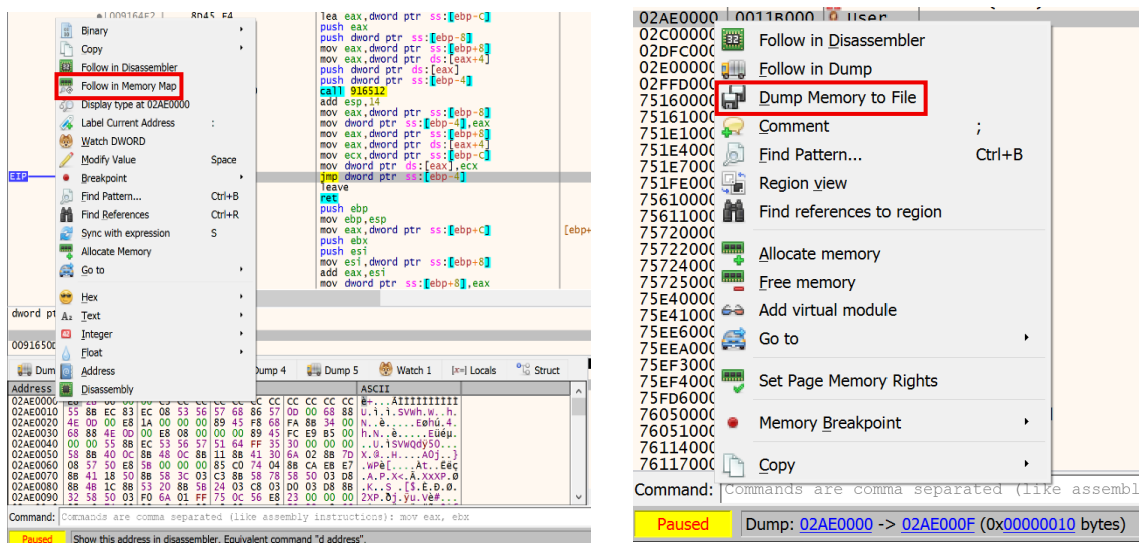
R: 16

Para verificar qué funciones se resuelven, se optó por extraer el shellcode y realizar un análisis dinámico mediante scdbg. Para ello, se estableció un breakpoint en la

instrucción donde se ejecuta el salto hacia el shellcode (jmp dword ptr ss:[ebp-4]) para interceptarlo instantes antes de ser ejecutado.

0091650B	8908	mov ecx, dword ptr ss:[ebp-4]
0091650D	FF65 FC	jmp dword ptr ds:[eax], ecx
00916510	C9	leave

Al depurar el malware se presentó un error de violación de acceso, lo que podría indicar la presencia de técnicas anti-debug; por ende, se utilizó el comando SHIFT+F9 para continuar la ejecución omitiendo las excepciones. Posteriormente, se realizó un seguimiento de la dirección de memoria asignada en el Memory Map y se extrajo el shellcode completo y desempaquetado.



Mediante scdbg, se determinó que el shellcode carga las librerías user32.dll, kernel32.dll y ntdll.dll. La librería kernel32 en específico, resuelve 16 funciones.

4016e8	LoadLibraryA(user32)
401702	GetProcAddress(MessageBoxA)
4017a8	GetProcAddress(GetMessageExtraInfo)
4017fa	LoadLibraryA(kernel32)
401831	GetProcAddress(WinExec)
40189f	GetProcAddress(CreateFileA)
4018de	GetProcAddress(WriteFile)
40194c	GetProcAddress(CloseHandle)
4019cf	GetProcAddress(CreateProcessA)
401a60	GetProcAddress(GetThreadContext)
401ad5	GetProcAddress(VirtualAlloc)
401b58	GetProcAddress(VirtualAllocEx)
401bc6	GetProcAddress(VirtualFree)
401c5e	GetProcAddress(ReadProcessMemory)
401cfd	GetProcAddress(WriteProcessMemory)
401d8e	GetProcAddress(SetThreadContext)
401e03	GetProcAddress(ResumeThread)
401ea9	GetProcAddress(WaitForSingleObject)
401f0c	GetProcAddress(GetModuleFileNameA)
401f96	GetProcAddress(GetCommandLineA)
401fef	LoadLibraryA(ntdll.dll)
402096	GetProcAddress(NtUnmapViewOfSection)

P8: The malware obfuscates two strings after calling RegisterClassExA. What is the first string?

R: saodkfnosa9uin

Para mapear la resolución de la función RegisterClassExA en x32dbg, se calculó el offset basándonos en la dirección base que utilizó scdbg (0x401000) y la dirección emulada de la función (4021cb), obteniendo el offset 11cb.

```
Using base offset: 0x401000
4016e8 LoadLibraryA(user32)
401702 GetProcAddress(MessageBoxA)
4017a8 GetProcAddress(GetMessageExtraInfo)
4017fa LoadLibraryA(kernel32)
401831 GetProcAddress(WinExec)
40189f GetProcAddress(CreateFileA)
4018de GetProcAddress(WriteFile)
40194c GetProcAddress(CloseHandle)
4019cf GetProcAddress(CreateProcessA)
401a60 GetProcAddress(GetThreadContext)
401ad5 GetProcAddress(VirtualAlloc)
401b58 GetProcAddress(VirtualAllocEx)
401bc6 GetProcAddress(VirtualFree)
401c5e GetProcAddress(ReadProcessMemory)
401cfd GetProcAddress(WriteProcessMemory)
401d8e GetProcAddress(SetThreadContext)
401e03 GetProcAddress(ResumeThread)
401ea9 GetProcAddress(WaitForSingleObject)
401f0c GetProcAddress(GetModuleFileNameA)
401f96 GetProcAddress(GetCommandLineA)
401fef LoadLibraryA(ntdll.dll)
402096 GetProcAddress(NtUnmapViewOfSection)
40213d GetProcAddress(NtWriteVirtualMemory)
4021cb GetProcAddress(RegisterClassExA)
402252 GetProcAddress(CreateWindowExA)
4022c4 GetProcAddress(PostMessageA)
402308 GetProcAddress(GetMessageA)
402388 GetProcAddress(DefWindowProcA)
```

Como la dirección de memoria base asignada para el shellcode en x32dbg es 02A90000, se determinó que la preparación de dicha función está ubicada en la dirección de memoria 02A911CB.

Address	Hex	ASCII
02A90000	E8 2B 06 00 C3 CC CC CC CC CC CC CC CC CC CC CC CC	e+...Äiiiiiiiiiii
02A90010	55 8B EC 83 EC 08 53 56 57 68 86 57 0D 00 68 88	U.i.i.SVWh.W..h.
02A90020	4E 0D 00 E8 1A 00 00 00 89 45 F8 68 FA 8B 34 00	N..è....Eøhú.4.
02A90030	68 88 4E 0D 00 E8 08 00 00 00 89 45 FC E9 B5 00	h.N..è....Eüém.
02A90040	00 00 55 8B EC 53 56 57 51 64 FF 35 30 00 00 00	..U.iSVWQdy50...
02A90050	58 8B 40 0C 8B 48 0C 8B 11 8B 41 30 6A 02 8B 7D	x. @. .H. . . .A0j. }
02A90060	08 57 50 E8 5B 00 00 00 85 C0 74 04 8B CA EB E7	.wPè[. . . .Ät. .Ëëç
02A90070	8B 41 18 50 8B 58 3C 03 C3 8B 58 78 58 50 03 D8	.A.P.X<.Ä.XxXP.ø
02A90080	8B 4B 1C 8B 53 20 8B 5B 24 03 C8 03 D0 03 D8 8B	.K..S.[\$.Ë.ø.ø.
02A90090	32 58 50 03 F0 6A 01 FF 75 0C 56 E8 23 00 00 00	2XP.øj.ÿu.vè#...

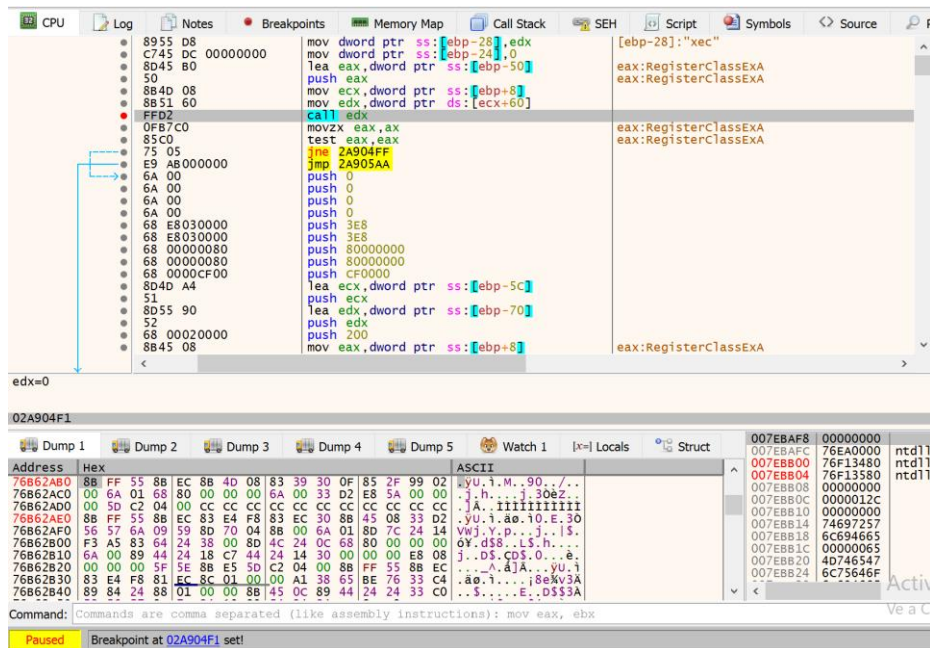
Posteriormente, se mapeó la dirección hallada en el visor Disassembler y se estableció un breakpoint para interceptar el instante en el que el malware almacena la dirección de la función RegisterClassExA en la variable [ebp-80].

```
EIP → 02A911C5 FF95 24FFFFFF call dword ptr ss:[ebp-DC] [ebp-DC]:GetProcAddress
02A911C6 8945 80 mov dword ptr ss:[ebp-80],eax [ebp-80]:&"PE", eax:RegisterClassExA
02A911CE C685 70DFFFFF 43 mov byte ptr ss:[ebp-290],43 43:'C'
02A911D5 C685 71EFFFFF 73 mov byte ptr ss:[ebp-28E],73 73:'s'
```

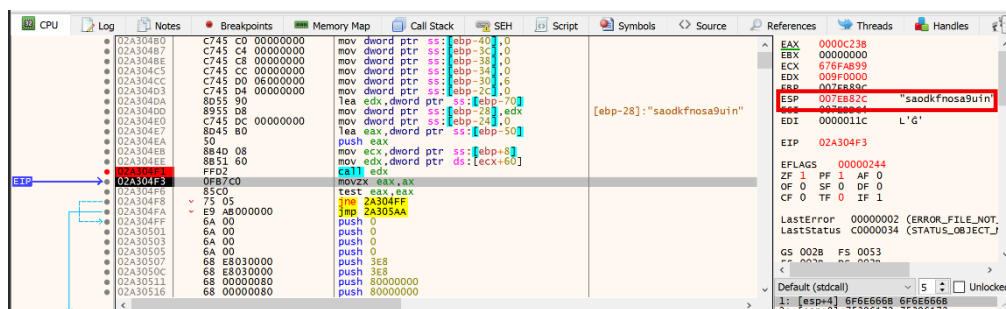
Como ya se confirmó que el malware sí obtiene la dirección de la función RegisterClassExA, se repitió el proceso esta vez para mapear el primer uso de dicha función teniendo el offset 4F1.

```
4015ee GetFileAttributesA(apfHQ)
4015ee GetFileAttributesA(apfHQ)
4014f1 unhooked call to user32.RegisterClassExA step=177419
```

Se realizó un seguimiento de la dirección 02A904F1 en el visor Disassembler y se estableció un breakpoint.



Se realizó un Step Over (F8) para visualizar el siguiente instante después del registro de la ventana en el sistema operativo, obteniendo la primera cadena ofuscada "saodkfnosa9uin", enviada a la función RegisterClassExA y perteneciente a la plantilla utilizada para la creación de la ventana en Windows.



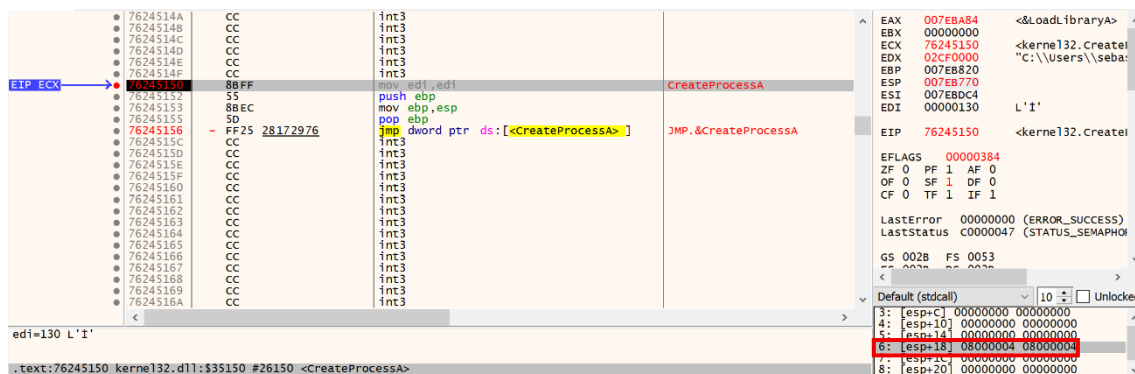
P9: What is the value of dwCreationFlags of CreateProcessA?

R: 0x08000004

Se estableció un breakpoint en la llamada a la API CreateProcessA. El sexto parámetro, dwCreationFlags, tiene un valor de 0x08000004, el cual representa la combinación (suma lógica) de dos banderas independientes:

- 0x00000004 (CREATE_SUSPENDED): Le ordena a Windows crear el proceso en memoria con su hilo principal suspendido, impidiendo la ejecución de cualquier instrucción hasta nuevo aviso.
- 0x08000000 (CREATE_NO_WINDOW): Le indica al sistema que el proceso creado debe ejecutarse oculto, sin desplegar una ventana de consola visible.

En conjunto, este comportamiento confirma que el malware crea un proceso invisible y suspendido con el propósito de activarlo en un momento durante el ataque, vaciar el código del proceso legítimo e inyectar el payload malicioso; es decir, el malware realiza una inyección de código dentro de un proceso legítimo del sistema.



P10: The Malware uses a process injection technique. What is its name?

R: Process Hollowing

La configuración del parámetro dwCreationFlags de la función CreateProcessA es una de las firmas compartidas por dos subtécnicas de la técnica Process Injection (T1055): Process Hollowing (T1055.012) y Dynamic-link Library Injection (T1055.001). Otra de las funciones clave que utilizan ambas variantes es VirtualAllocEx, la cual se confirmó previamente que el malware emplea para interactuar con la memoria del proceso remoto.

La subtécnica T1055.012, a diferencia de la T1055.001, vacía por completo el código del proceso objetivo utilizando la función `NtUnmapViewOfSection` o `ZwUnmapViewOfSection`. Por el contrario, la subtécnica T1055.001 depende de la función `CreateRemoteThread` con el objetivo de crear un hilo secundario dentro del proceso objetivo para cargar una biblioteca.

La Tabla de Direcciones de Importación (IAT) obtenida mediante `scdbg` demuestra que la subtécnica empleada por el malware es Process Hollowing (T1055.012), ya que el binario importa la función `NtUnmapViewOfSection` y carece por completo de la función `CreateRemoteThread`.

```
C:\> Administrador: Símbolo del sistema
Using base offset: 0x401000

4016e8  LoadLibraryA(user32)
401702  GetProcAddress(MessageBoxA)
4017a8  GetProcAddress(GetMessageExtraInfo)
4017fa  LoadLibraryA(kernel32)
401831  GetProcAddress(WinExec)
40189f  GetProcAddress(CreateFileA)
4018de  GetProcAddress(WriteFile)
40194c  GetProcAddress(CloseHandle)
4019cf  GetProcAddress(CreateProcessA)
401a60  GetProcAddress(GetThreadContext)
401ad5  GetProcAddress(VirtualAlloc)
401b58  GetProcAddress(VirtualAllocEx)
401bc6  GetProcAddress(VirtualFree)
401c5e  GetProcAddress(ReadProcessMemory)
401cfd  GetProcAddress(WriteProcessMemory)
401d8e  GetProcAddress(SetThreadContext)
401e03  GetProcAddress(ResumeThread)
401ea9  GetProcAddress(WaitForSingleObject)
401f0c  GetProcAddress(GetModuleFileNameA)
401f96  GetProcAddress(GetCommandLineA)
401fef  LoadLibraryA(ntdll.dll)
402096  GetProcAddress(NtUnmapViewOfSection)
40213d  GetProcAddress(NtWriteVirtualMemory)
4021cb  GetProcAddress(RegisterClassExA)
402252  GetProcAddress(CreateWindowExA)
4022c4  GetProcAddress(PostMessageA)
402308  GetProcAddress(GetMessageA)
402388  GetProcAddress(DefWindowProcA)
4023e8  GetProcAddress(GetFileAttributesA)
40246f  GetProcAddress(GetStartupInfoA)
4024fd  GetProcAddress(VirtualProtectEx)
402568  GetProcAddress(ExitProcess)
4015ee  GetFileAttributesA(apfHQ)
4015ee  GetFileAttributesA(apfHQ)
4015ee  GetFileAttributesA(apfHQ)
4014f1  unhooked call to user32.RegisterClassExA      step=177419
```


P11: What is the API used to write the payload into the target process?

R: WriteProcessMemory

Dado que se identificó la subtécnica específica empleada, se utilizó la documentación de dicha subtécnica dada por MITRE ATT&CK. Se confirmó nuevamente el flujo que sigue el malware y las APIs que utiliza. La API que utilizó para la escritura del payload en el proceso legítimo es WriteProcessMemory.

MITRE | ATT&CK

Matrices | Tactics | Techniques | Defenses | CTI | Resources | Benefactors | Contribute | Blog

TECHNIQUES

ListPlanting

Scheduled Task/Job

Valid Accounts

Stealth

Defense Impairment

Credential Access

Discovery

Lateral Movement

Collection

Process Injection: Process Hollowing

Other sub-techniques of Process Injection (12)

Adversaries may inject malicious code into suspended and hollowed processes in order to evade process-based defenses. Process hollowing is a method of executing arbitrary code in the address space of a separate live process.

Process hollowing is commonly performed by creating a process in a suspended state then unmapping/hollowing its memory, which can then be replaced with malicious code. A victim process can be created with native Windows API calls such as `CreateProcess`, which includes a flag to suspend the processes primary thread. At this point the process can be unmapped using APIs calls such as `ZwUnmapViewOfSection` or `NtUnmapViewOfSection` before being written to, realigned to the injected code, and resumed via `VirtualAllocEx`, `WriteProcessMemory`, `SetThreadContext`, then `ResumeThread` respectively.^{[1][2]}

ID: T1055.012

Sub-technique of: T1055

Tactics: Stealth, Privilege Escalation

Platforms: Windows

Version: 2.0

Created: 14 January 2020

Last Modified: 12 May 2026

Version Permalink

Se confirmó mediante x32dbg el uso de la API WriteProcessMemory.

EDX	762475FF	CC	int3	
	76247600	8BFF	mov edi,edi	writeProcessMemory
	76247602	55	push ebp	
	76247603	8BEC	mov ebp,esp	
	76247605	5D	pop ebp	
EIP	76247608	FF25 68152976	jmp dword ptr ds:[<writeProcessMemory>]	JMP.&writeProcessMemory
	7624760C	CC	int3	